



Facultad de Ciencias

DASHBOARD PARA EVALUAR LA EXPERIENCIA DE USUARIO EN APLICACIONES DE INTELIGENCIA ARTIFICIAL INTERACTIVAS

(DASHBOARD FOR EVALUATING USER EXPERIENCE IN
INTERACTIVE ARTIFICIAL INTELLIGENCE APPLICATIONS)

Trabajo de Fin de Grado
para acceder al

GRADO EN INGENIERÍA INFORMÁTICA

Autor: Lucas González Puente

Director: Rafael Duque Medina

Julio - 2026

Agradecimientos

Quiero, en primer lugar, agradecer a mi director Rafael Duque por la implicación que ha tenido en el desarrollo del trabajo. Por poner empeño y dedicación en crear una herramienta sólida y de calidad. Por ir más allá de la dirección del proyecto, mostrando interés por mi desarrollo en la carrera y brindándome consejos para el futuro.

Agradecer también a mi familia, mi ejemplo a seguir, por invertir en mi futuro no solo con herramientas sino con un apoyo incondicional en los momentos de duda a lo largo de estos 4 años.

A mis amigos, simplemente por estar ahí. Las charlas interminables y los planes improvisados siempre han sido el refugio perfecto.

A Lucía, que ha sabido acompañarme a pesar de los kilómetros que nos separan. Gracias por sostenerme siempre y celebrar cada avance como si fuese tuyo.

A todos, muchas gracias.

Lucas González Puente, Pedreña, 2026.

Resumen

Hoy en día la Inteligencia Artificial está presente en muchas decisiones importantes de nuestra vida, desde recomendaciones de rutas hasta diagnósticos médicos. Sin embargo, que un sistema de IA "funcione bien" no depende solo de que sus predicciones sean correctas: también importa si la persona que lo usa lo entiende, si confía en él de forma razonable y si le resulta cómodo de utilizar. Este lado humano de la IA muchas veces no se mide, y las empresas que desarrollan estos sistemas no siempre tienen una forma sencilla de comprobarlo.

Este Trabajo de Fin de Grado consiste en el desarrollo de una herramienta web pensada para cubrir ese hueco. La plataforma permite evaluar cualquier sistema de IA combinando dos tipos de información: por un lado, datos objetivos sobre cómo ha interactuado la persona con el sistema, recogidos de forma automática; y por otro lado, datos subjetivos sobre cómo se ha sentido esa persona durante la prueba mediante un cuestionario sencillo. Toda esa información se cruza y se muestra en un panel de control, pensado para que un evaluador pueda ver de un vistazo si existe un desajuste entre lo que el sistema consigue y lo que la persona percibe: por ejemplo, detectar casos en los que alguien confía demasiado en una IA poco fiable, o casos en los que desconfía de una IA que en realidad funciona bien.

Palabras clave: Inteligencia Artificial Centrada en el Ser Humano (HCAI), Experiencia de Usuario, Evaluación de Sistemas, Confianza, Panel de Control (Dashboard).

Abstract

Today, Artificial Intelligence is present in many important decisions in our lives, from route recommendations to medical diagnoses. However, an AI system "working well" doesn't just depend on whether its predictions are correct: it also matters whether the person using it understands it, whether they trust it in a reasonable way, and whether they find it comfortable to use. This human side of AI is often not measured, and the companies developing these systems don't always have a simple way to check it.

This Final Degree Project consists of developing a web tool designed to fill that gap. The platform allows any AI system to be evaluated by combining two types of information: on one hand, objective data about how the person interacted with the system, collected automatically; and on the other hand, subjective data about how that person felt during the test, collected through a simple questionnaire. All this information is cross-checked and displayed on a dashboard, designed so that an evaluator can see at a glance whether there's a mismatch between what the system achieves and what the person perceives: for example, detecting cases where someone trusts an unreliable AI too much, or cases where someone distrusts an AI that actually works well.

Keywords: Human-Centered Artificial Intelligence (HCAI), User Experience, System Evaluation, Trust, Dashboard.

Índice general

1. Introducción	9
1.1. Motivación	9
1.2. Contexto Tecnológico	11
1.3. Objetivos	11
1.4. Estructura del documento	12
2. Análisis de requisitos	13
2.1. Requisitos funcionales	13
2.2. Requisitos no funcionales	15
3. Metodología y Herramientas	16
3.1. Metodología	16
3.1.1. Primera iteración: Mecanismo de apoyo para que los usuarios del sistema bajo prueba evalúen aspectos HCAI	16
3.1.2. Segunda iteración: Procesamiento de logs y el Dashboard visual	17
3.1.3. Tercera iteración: Seguridad, Pruebas, Refinamiento y Experiencia de Usuario	17
3.2. Organización temporal	18
3.3. Herramientas	20
3.3.1. Frontend: React y Recharts	20
3.3.2. Backend: Python y FastAPI	20
3.3.3. Persistencia de Datos: PostgreSQL y SQLAlchemy	20
3.3.4. Entorno de Desarrollo y Gestión del Proyecto	21
4. Diseño y Construcción	22
4.1. Arquitectura del sistema	22
4.2. Modelo de datos	23
4.3. Estructura del frontend React	24
4.4. Interfaz del participante	25
4.4.1. Pantalla de selección e inicio de sesión	25
4.4.2. Encuesta multi-paso	26
4.4.3. Estructura del archivo de log	30
4.4.4. Procesamiento en el servidor	30
4.5. Interfaz del evaluador	31
4.5.1. Autenticación y control de acceso	31
4.5.2. Panel de configuración de experimentos	31

4.5.3.	Dashboard de análisis	32
4.5.4.	Cálculo del HCAI Score	32
5.	Evaluación	34
5.1.	Suite de pruebas automatizadas	34
5.1.1.	Infraestructura de pruebas	34
5.1.2.	Módulos de prueba	34
5.1.3.	Resumen de la suite	36
5.2.	Prueba de uso con aplicación real: Predictor de Diabetes	36
5.2.1.	Selección y descripción de la aplicación bajo prueba	36
5.2.2.	Configuración del experimento en la plataforma	37
5.2.3.	Flujo de evaluación seguido por el participante	38
5.2.4.	Verificación de los resultados en el dashboard	39
5.2.5.	Conclusiones de la prueba	40
6.	Conclusiones	41
6.1.	Logros alcanzados	41
6.2.	Grado de cumplimiento de los objetivos	41
6.3.	Líneas de trabajo futuro	42
A.	Lista de Acrónimos	44
	Bibliografía	44

Índice de figuras

1.1.	Google Maps presentando varias rutas con información de tráfico, peajes y tiempo estimado, dejando la decisión final al usuario.	10
3.1.	Diagrama de Gantt del Trabajo de Fin de Grado	19
4.1.	Esquema del árbol de directorios con los archivos principales.	23
4.2.	Diagrama Entidad-Relación de la base de datos hcai_db.	24
4.3.	Ejemplo de pregunta incluida en la sección de Confianza.	26
5.1.	Interfaz de usuario del sistema de predicción de diabetes	37
5.2.	Panel de configuración de experimentos: definición de la prueba <i>E-DiaB</i> con el Ground Truth introducido por el evaluador.	38
5.3.	Pantalla de bienvenida del participante, mostrando la tarea asignada para la prueba <i>E-DiaB</i>	39
5.4.	Métricas subjetivas por sesión en el dashboard: evolución de Confianza, Explicabilidad y Carga Cognitiva a lo largo de las sesiones registradas. . .	39

Índice de tablas

2.1.	Relación de Requisitos Funcionales	14
2.2.	Relación de Requisitos No Funcionales.	15
4.1.	Ítems de la dimensión Confianza (escala Likert 1–5). Adaptados de Hoffman et al. [1]	28
4.2.	Escala de respuesta Likert empleada en las secciones 1 y 2.	28
4.3.	Ítems de la dimensión Explicabilidad.	29
4.4.	Ítems de la dimensión Carga Cognitiva, adaptados de la escala NASA-TLX [2]	29
4.5.	Normalización de la escala Likert.	33
5.1.	Resumen de la suite de pruebas automatizadas.	36

Capítulo 1

Introducción

La integración de la Inteligencia Artificial (IA) en productos digitales ha revolucionado la forma en la que los humanos interactúan con la tecnología. Mientras que las métricas clásicas se centran, por ejemplo, en la capacidad predictiva y en el rendimiento del sistema, el éxito de una aplicación basada en IA radica también en aspectos centrados en las personas como, por ejemplo, la facilidad para comprender las decisiones del sistema y la confianza del usuario, fluidez de la conversación y comportamiento ante errores. Este capítulo detalla los fundamentos que justifican el desarrollo del trabajo, explorando la necesidad de equilibrar el rendimiento técnico con la experiencia del usuario.

1.1. Motivación

En el panorama tecnológico actual, la Inteligencia Artificial se ha consolidado no solo como una herramienta de innovación, sino como un agente de cambio equiparable a revoluciones industriales precedentes. La trascendencia de su uso ha logrado redefinir los paradigmas de productividad y optimizar la eficiencia operativa.

Aunque la capacidad de automatización y análisis de datos define y demuestra la potencia técnica de la IA, parte de su trascendencia también reside en el paradigma de la IA Centrada en el Ser Humano. Bajo este enfoque, la tecnología abandona el rol de mero sustituto para convertirse en un instrumento para potenciar las capacidades, el bienestar y toma de decisiones de las personas.

El objetivo fundamental de la HCAI (Human-Centered Artificial Intelligence) es aumentar las capacidades humanas manteniendo el control humano sobre los sistemas, considerando no solo la necesidad técnica, sino también el contexto, las condiciones éticas y legales, y la promoción del bienestar individual y social [3].

Esto implica que el desarrollo de la IA no puede limitarse al algoritmo, sino que debe integrar tres pilares fundamentales: (i) que el usuario mantenga el control, (ii) que el sistema sea comprensible y (iii) que actúe de forma ética y responsable, todo ello bajo un proceso de diseño iterativo que involucre al usuario.

Un ejemplo de esta relación humano-algoritmo es Google Maps. Mediante el uso de IA, esta plataforma trabaja con multitud de variables en tiempo real como tráfico, peajes, obras... y las sintetiza en un conjunto de opciones presentadas al conductor de forma comprensible. El sistema no reemplaza la decisión humana: sugiere la ruta más rápida, la más económica o la que evita autopistas de peaje, pero es el conductor quien elige (véase Figura 1.1). Este comportamiento es precisamente el que define la HCAI: la IA actúa como agente que reduce la incertidumbre amplificando la capacidad de decisión del humano en lugar de suplantarla.

Este tipo de sistemas genera además un conjunto de datos que permiten evaluar la calidad de esa interacción humano-IA: la *tasa de aceptación de recomendaciones* (¿con qué frecuencia el usuario sigue la ruta sugerida?), la *tasa de rechazo* (¿cuántas veces la descarta?), el *número de correcciones manuales* (¿cuántas veces el usuario ignora la recalculación automática?) y el *uso de las explicaciones* que el sistema ofrece para justificar su propuesta. Son exactamente este tipo de métricas las que la herramienta desarrollada en este trabajo aspira a recolectar y visualizar.

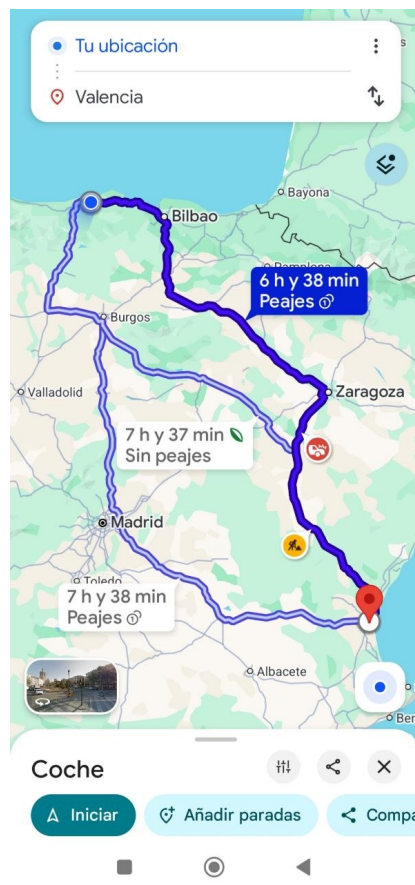


Figura 1.1: Google Maps presentando varias rutas con información de tráfico, peajes y tiempo estimado, dejando la decisión final al usuario.

Sin embargo, la adopción del paradigma HCAI plantea el desafío de cómo evaluar correctamente estos sistemas. Definir métricas de evaluación apropiadas para sistemas HCAI es una cuestión compleja, principalmente porque una evaluación adecuada debe realizarse a lo largo de dos dimensiones: el rendimiento técnico y los resultados centrados en el ser humano. Las métricas de evaluación tradicionales de la IA, como la exactitud (accuracy), la precisión y la exhaustividad (recall), ponen un foco singular en el algoritmo [4].

No obstante, estas métricas a menudo ignoran el contexto más amplio de la interacción humano-IA, dejando de lado factores determinantes como la satisfacción del usuario, la confianza y la efectividad de la IA para apoyar la toma de decisiones humana. Por tanto, existe la necesidad de establecer medidas de rendimiento holísticas que combinen estas dimensiones centradas en el humano —posiblemente integrando retroalimentación cualitativa de los usuarios, métricas relacionadas con la carga cognitiva y medidas de comprensión— con la robustez técnica.

En respuesta a esta problemática, este trabajo propone la creación de una herramienta que proporcione soporte para ambas dimensiones. El objetivo es facilitar una evaluación integral que no solo valide la eficacia técnica de los modelos, sino que también garantice que estos cumplan con los estándares de usabilidad, confianza y alineación con los principios exigidos por el paradigma HCAI.

1.2. Contexto Tecnológico

Bajo estas premisas, este trabajo se centra en el desarrollo de una solución basada en una web, diseñada para hacer que la participación de usuarios y evaluadores sea lo más accesible posible durante el proceso de evaluación. La recolección de métricas requiere que el software de soporte desarrollado interfiera lo menos posible en la experiencia de usuario; de lo contrario, la complejidad de la herramienta podría sesgar los resultados.

Por este motivo, se ha optado por una solución accesible a través del navegador. Esta elección elimina la barrera de entrada que suponen las instalaciones de software específico; así los usuarios que pueden participar en las pruebas, desde sus propios dispositivos y en su entorno natural, facilitando una participación más amplia.

1.3. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es diseñar e implementar una herramienta software con un dashboard interactivo que permita evaluar el grado de cumplimiento de los principios de la HCAI en sistemas inteligentes.

La herramienta combinará el análisis automático de archivos de log —evidencia objetiva que recoge métricas de comportamiento como el número de errores cometidos, los tiempos de uso, las interacciones realizadas o el número de correcciones manuales sobre las sugerencias de la IA— con resultados de encuestas a usuarios (percepción subjetiva) para ofrecer un diagnóstico integral del sistema evaluado.

Para alcanzar el objetivo global, se han definido los siguientes objetivos parciales:

- **Definición de un conjunto de métricas HCAI:** Establecer un marco teórico y práctico de indicadores que combinen datos cuantitativos y cualitativos para medir dimensiones clave como la confianza, la carga cognitiva y la transparencia del usuario.
- **Desarrollo de un sistema de ingesta y procesamiento de datos:** Diseñar un backend robusto capaz de recibir datos de sistemas externos y almacenar las respuestas de los usuarios en una base de datos.
- **Implementación de visualización interactiva:** Crear un Dashboard que transforme los datos brutos en información comprensible mediante gráficos, líneas de tendencia, comparativas, etc.
- **Evaluación de la herramienta:** Validar la utilidad del programa mediante pruebas de uso. Ej. Overtrust/Undertrust

1.4. Estructura del documento

La presente memoria se estructura en seis capítulos principales, organizados de forma lógica para guiar al lector desde la concepción de la idea hasta la validación final del software desarrollado:

- **Introducción.** Presenta la motivación del proyecto en el contexto de la HCAI, define los objetivos principales y parciales, y describe la estructura general del documento.
- **Análisis de requisitos.** Detalla las necesidades del sistema extraídas de los objetivos, clasificándolas en requisitos funcionales y no funcionales.
- **Metodologías y Herramientas.** Describe el ciclo de vida iterativo adoptado para la gestión del proyecto y justifica técnica y académicamente el conjunto de tecnologías elegidas para el desarrollo.
- **Diseño y Construcción.** Constituye el núcleo técnico del trabajo. En él se detalla la arquitectura del sistema, el modelado de la base de datos, el diseño de la interfaz de usuario y la implementación algorítmica de la lógica de negocio.
- **Evaluación.** Expone las pruebas de validación realizadas sobre el sistema para garantizar que la recolección, el cruce de datos objetivos y subjetivos, y su posterior visualización en el dashboard son correctos y precisos.
- **Conclusiones.** Resume los logros alcanzados, evalúa el grado de cumplimiento de los objetivos propuestos inicialmente y plantea posibles líneas de trabajo futuro para la mejora de la herramienta.

Capítulo 2

Análisis de requisitos

A continuación, se detallan los requisitos funcionales y no funcionales identificados para el desarrollo de la herramienta, priorizados según su importancia para el producto mínimo funcional.

El sistema contempla dos tipos de usuarios con roles claramente diferenciados:

- **Evaluador:** Responsable de la evaluación. Accede mediante autenticación al panel de configuración y al *dashboard*. Su función es diseñar las pruebas, introducir las métricas técnicas del sistema bajo evaluación y analizar los resultados agregados.
- **Usuario del sistema bajo prueba:** Participante que accede de forma pública a la interfaz de encuesta y selecciona el experimento que va a evaluar. A través de esta interfaz, lee la descripción de la tarea asignada, la cual ha sido configurada previamente por el evaluador para informarle detalladamente sobre las acciones precisas que debe realizar con el sistema externo. A continuación, realiza la prueba y, al finalizar, completa el cuestionario y sube el archivo de log generado por dicho sistema.

2.1. Requisitos funcionales

Un requisito funcional describe lo que el sistema debe hacer. Establece el comportamiento observable del sistema ante una entrada o evento. En la Tabla 2.1 se desarrollan los requisitos funcionales.

ID	Requisito	Descripción	Usuario	Prio.
RF01	Selección de prueba	El usuario debe poder seleccionar el sistema que va a evaluar y visualizar la descripción de la tarea asignada.	Participante	Alta
RF02	Encuesta subjetiva	El usuario debe poder responder un cuestionario estructurado en: confianza, explicabilidad y carga cognitiva.	Participante	Alta
RF03	Subida de log de interacción	El usuario debe poder adjuntar el archivo JSON de eventos generado por el sistema bajo prueba.	Participante	Alta
RF04	Configuración de experimentos	El evaluador debe poder definir nuevas pruebas introduciendo los campos pertinentes (nombre, descripción). Cada prueba genera un token único y un enlace de acceso.	Evaluador	Alta
RF05	Autenticación del evaluador	El acceso al <i>dashboard</i> y al panel de configuración debe estar protegido mediante un sistema de autenticación por credenciales.	Evaluador	Alta
RF06	Dashboard de resultados	El evaluador debe poder visualizar un panel de control con distintas métricas obtenidas de la aplicación, todo ello cruzado por sesión.	Evaluador	Alta
RF07	Evaluación temporal y ranking	El evaluador debe poder visualizar el histórico completo de sesiones, así como un ranking de participantes ordenado por puntuación HCAI para identificar tendencias y valores atípicos.	Evaluador	Media
RF08	Diagnóstico de calibración	El <i>dashboard</i> debe mostrar el gap de calibración entre la confianza percibida por el usuario y la tasa de éxito derivada del log de interacción.	Evaluador	Media
RF09	Generación de informe	El evaluador debe poder exportar un informe completo con todos los resultados.	Evaluador	Media

Tabla 2.1: Relación de Requisitos Funcionales

2.2. Requisitos no funcionales

En cuanto a los requisitos no funcionales, estos describen cómo debe comportarse el sistema, estableciendo restricciones de calidad sobre su arquitectura, rendimiento, seguridad y mantenibilidad. En la Tabla 2.2 se recogen las restricciones identificadas por la herramienta, clasificadas por categoría.

ID	Requisito	Descripción	Categoría
RNF01	Arq. Cliente-Servidor	El sistema debe seguir una arquitectura cliente-servidor separando el frontend del backend.	Arquitectura
RNF02	Base de Datos	Los datos deben almacenarse en una base de datos relacional.	Almacenamiento
RNF03	Privacidad	El sistema no debe almacenar datos personales identificables, utilizando identificadores únicos [5].	Seguridad
RNF04	Tiempo de respuesta	Las visualizaciones del dashboard deben cargar en un tiempo aceptable para el usuario.	Rendimiento
RNF05	Usabilidad	La interfaz debe ser intuitiva, permitiendo completar las encuestas en menos de 2 minutos.	Usabilidad
RNF06	Modularidad	El backend debe permitir añadir nuevas métricas HCAI sin reescribir la lógica de ingesta.	Mantenibilidad
RNF07	Portabilidad	Aplicación web compatible con versiones recientes de Chrome, Firefox y Safari.	Portabilidad
RNF08	Integridad de datos	Los logs de IA deben vincularse unívocamente a las encuestas mediante tokens de sesión.	Fiabilidad

Tabla 2.2: Relación de Requisitos No Funcionales.

Capítulo 3

Metodología y Herramientas

3.1. Metodología

Para el desarrollo del trabajo se ha adoptado un ciclo de vida iterativo e incremental [6]. Este enfoque se caracteriza por descomponer el proyecto en pequeños bloques de funcionalidad que se construyen de manera progresiva y puede refinarse sucesivamente.

Esta elección fue tomada por la naturaleza de las métricas HCAI: al tratarse de un campo donde la experiencia de usuario es subjetiva, es fundamental poder crear interfaces, validarlas y ajustarlas sin necesidad de completar todo el sistema en una fase.

El desarrollo se ha planificado en iteraciones funcionales, permitiendo disponer de una versión operativa al final de cada ciclo.

3.1.1. Primera iteración: Mecanismo de apoyo para que los usuarios del sistema bajo prueba evalúen aspectos HCAI

La primera iteración del desarrollo se ha centrado exclusivamente en los Requisitos Funcionales relacionados con el usuario del sistema bajo prueba (el participante que realiza la evaluación).

El objetivo principal de esta fase inicial ha sido garantizar la correcta recolección de los datos subjetivos, siendo estos los que constituyen la base necesaria para cualquier análisis posterior. Sin una interfaz de captura robusta y amigable, el resto del sistema carece de utilidad. En concreto, esta iteración ha abarcado las siguientes tareas y funcionalidades:

- **Implementación del Frontend de Encuestas:** Desarrollo de la interfaz para presentar los cuestionarios de confianza y explicabilidad.
- **Integración de escalas especializadas:** Desarrollo de componentes interactivos (deslizadores) para la escala NASA-TLX, asegurando una interacción fluida y precisa. NASA-TLX se define como una herramienta de evaluación subjetiva de la carga de trabajo. Esta herramienta permite realizar evaluaciones subjetivas sobre la carga de trabajo de los operadores que interactúan con diversos sistemas de interfaz persona-computador [2].

- **Comunicación Cliente-Servidor:** Configuración inicial del Backend (FastAPI) y la base de datos relacional para asegurar que las respuestas enviadas por el usuario se persisten correctamente en el servidor. En esta fase se optó inicialmente por SQLite como motor de base de datos por su inmediatez y ausencia de configuración, lo que permitió acelerar el ciclo de prototipado. Sin embargo, sus limitaciones en cuanto a concurrencia y ausencia de integridad referencial completa lo hicieron inviable para un sistema que debía relacionar varias entidades y recibir escrituras simultáneas de múltiples participantes. Esta restricción motivó la migración a PostgreSQL.

Al finalizar esta iteración, el sistema ya permitía a un usuario completar el flujo de evaluación completo, desde la lectura de las preguntas hasta la confirmación de envío, sentando las bases para la segunda iteración.

3.1.2. Segunda iteración: Procesamiento de logs y el Dashboard visual

En esta fase se implementó el motor de procesamiento de logs y el Dashboard visual. El enfoque del dashboard permite al sistema contrastar la realidad técnica del modelo evaluado con la experiencia reportada por los usuarios participantes. Para ello, se desarrolló en primer lugar una interfaz orientada al evaluador que permite definir el *Ground Truth* del modelo de IA, introduciendo las métricas teóricas esperadas, como la exactitud (*Accuracy*) o el valor *F1-Score*.

Paralelamente, se crearon los esquemas y *endpoints* en el servidor FastAPI necesarios para la ingesta continua de datos derivados de la interacción del usuario, tales como el tiempo de resolución de la tarea, el número de clics y errores cometidos. Tras asegurar la recolección de los datos, la integración de la librería *Recharts* posibilitó la construcción de un panel de control interactivo con gráficos como LineChart, BarChart o ScatterChart.

Al concluir este ciclo, la herramienta ya era capaz de cruzar la evidencia objetiva con la percepción subjetiva, cumpliendo así con el objetivo principal de la investigación.

3.1.3. Tercera iteración: Seguridad, Pruebas, Refinamiento y Experiencia de Usuario

La última fase del desarrollo se centró en dotar al sistema de características propias de un producto de software profesional, mejorando la robustez, su seguridad y refinando la usabilidad general de las interfaces. En primer lugar, se llevó a cabo la migración desde la base de datos ligera inicial (SQLite) hacia el gestor relacional PostgreSQL, garantizando una mayor integridad referencial y escalabilidad a largo plazo. Vinculado a esto, las respuestas Likert se normalizan a una escala 0–100 mediante la tabla de conversión definida en `hcai_logic.py`, permitiendo su integración directa en el cálculo del HCAI Score junto con los valores de la escala NASA-TLX, que ya se recogen en ese mismo rango mediante los deslizadores de la encuesta.

En el ámbito de la seguridad y el control de acceso, se incorporó un sistema de autenticación basado en JWT. Este mecanismo protege el panel de investigación y los resultados

agregados, asegurando que únicamente los evaluadores tengan acceso a los datos sensibles, mientras que la interfaz de encuesta permanece accesible de forma pública para los participantes.

Por otro lado, para optimizar la *UX*, se reestructuró el formulario de evaluación transformándolo en un asistente guiado de múltiples pasos (*wizard*). Este diseño progresivo, apoyado por técnicas de diseño de interfaces modernas como el *Glassmorphism* y el uso de transiciones asíncronas, persigue el objetivo de minimizar la carga cognitiva y el sesgo de fatiga del participante durante la ejecución de la prueba. Finalmente, el análisis visual se complementó con una tabla de datos (*Data Grid*) en el panel de control, la cual desglosa las métricas sesión por sesión, haciendo que el usuario tenga una capacidad de auditoría a nivel de usuario individual.

3.2. Organización temporal

La planificación temporal del proyecto se ha estructurado en fases secuenciales que reflejan el ciclo de vida iterativo e incremental descrito en las secciones anteriores. Esta organización ha permitido facilitar la adaptación ante posibles imprevistos, tal y como se refleja en el diagrama de Gantt (véase Figura 3.1).

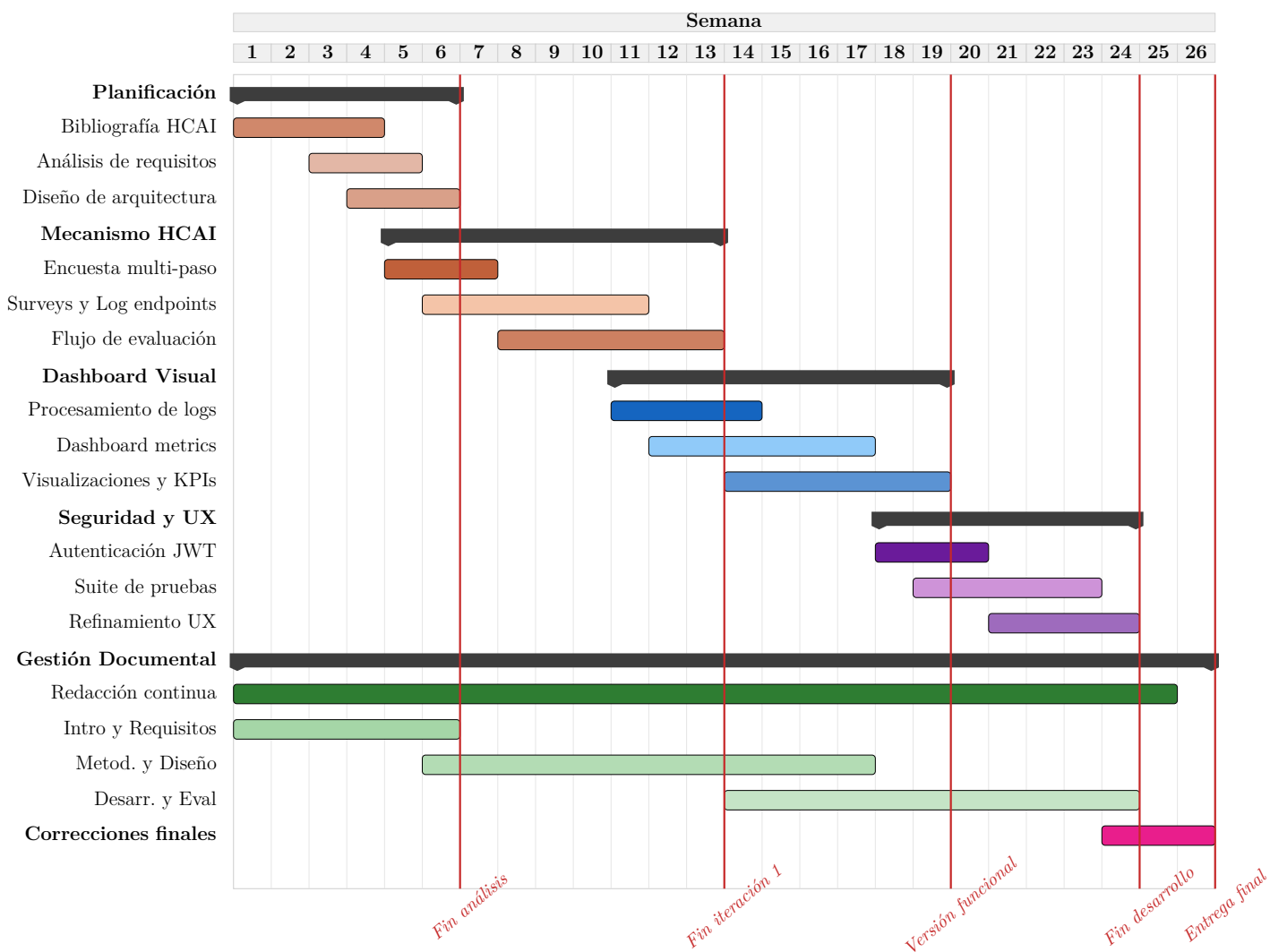


Figura 3.1: Diagrama de Gantt del Trabajo de Fin de Grado

3.3. Herramientas

Para la implementación de la plataforma se ha optado por un *stack* tecnológico moderno, orientado a la creación de aplicaciones web escalables y eficientes. A continuación, se justifican las tecnologías principales utilizadas en el desarrollo en sus tres capas fundamentales: la interfaz de usuario, la lógica de servidor y la persistencia de datos.

3.3.1. Frontend: React y Recharts

Para el desarrollo de la interfaz de usuario se ha utilizado la librería React [7]. Esta decisión técnica se fundamenta principalmente en su arquitectura basada en componentes, la cual permite dividir la interfaz en piezas independientes y reutilizables. Esta característica ha resultado crítica para la implementación de los cuestionarios, donde elementos como tarjetas de preguntas o los deslizadores de la escala NASA-TLX se definen una única vez y se instancian dinámicamente según sea necesario. Además, React facilita una gestión eficiente del estado de la aplicación, permitiendo controlar la lógica de navegación en el formulario multi-paso y la persistencia temporal de las respuestas.

Para completar la interfaz, especialmente en el apartado del cuadro de mandos (*dashboard*), se ha integrado **Recharts**. Esta librería de visualización de datos ha sido fundamental para implementar gráficos interactivos, destacando el *BarChart* de triangulación que permite superponer y comparar visualmente el rendimiento de la IA frente a la confianza percibida por el usuario de forma intuitiva.

3.3.2. Backend: Python y FastAPI

El servidor encargado de procesar la lógica de negocio y proveer los datos mediante una API REST ha sido desarrollado en **Python**, empleando el entorno de trabajo **FastAPI**. La elección de este *framework* obedece a su alto rendimiento y a su diseño para manejar peticiones asíncronas, garantizando tiempos de carga reducidos en el *dashboard* incluso al procesar un gran volumen de encuestas y registros de uso (*logs*).

Asimismo, FastAPI integra la validación de datos a través de la librería *Pydantic*, lo que asegura que toda la información entrante cumpla estrictamente con los esquemas predefinidos, rechazando peticiones malformadas de forma automática. Adicionalmente, el uso de Python en el *backend* sienta una base sólida para el proyecto de cara a futuras integraciones con librerías del ecosistema científico y de análisis de datos, tales como *pandas* o *skicit-learn*.

3.3.3. Persistencia de Datos: PostgreSQL y SQLAlchemy

Finalmente, el almacenamiento de la información generada se ha confiado a **PostgreSQL**, un sistema de gestión de bases de datos relacional robusto. Su elección se justifica por su excelente capacidad para relacionar las distintas entidades del sistema (encuestas, registros técnicos y configuraciones de las pruebas) y por su soporte nativo para el formato JSON, una característica ideal para almacenar las respuestas de los usuarios al poseer una estructura de longitud variable.

Para gestionar la interacción entre el servidor Python y la base de datos, se ha implementado **SQLAlchemy** como herramienta ORM. Este componente abstrae la complejidad de las sentencias SQL puras, permitiendo tratar las tablas como clases en el código y previniendo de manera activa vulnerabilidades de seguridad.

3.3.4. Entorno de Desarrollo y Gestión del Proyecto

De forma complementaria al *stack* tecnológico principal, el ciclo de construcción del software se ha apoyado en un conjunto de herramientas estándar de la industria. Como IDE principal se ha utilizado **Visual Studio Code**, seleccionado por su ligereza, versatilidad y amplio abanico de extensiones, tanto para React como para Python.

Para el control de versiones, se ha empleado **Git** en conjunto con la plataforma **GitHub**, lo que ha garantizado un registro trazable y seguro de las distintas iteraciones del proyecto descritas anteriormente. Finalmente, la administración, monitorización y consulta directa de la base de datos PostgreSQL durante la fase de construcción y pruebas se han llevado a cabo de manera ágil mediante la interfaz gráfica **pgAdmin**.

Con el objetivo de simplificar la puesta en marcha de la aplicación y garantizar que funcione correctamente en cualquier máquina, se ha utilizado **Docker**. Esta herramienta permite ejecutar todos los componentes del sistema (base de datos, backend y frontend) de forma conjunta y aislada, sin necesidad de instalar manualmente ninguna dependencia en el equipo del usuario.

Gracias a esta configuración, cualquier persona puede ejecutar la aplicación completa con un único comando, independientemente de su sistema operativo o entorno de trabajo.

Capítulo 4

Diseño y Construcción

Este capítulo describe las decisiones de diseño tomadas durante el desarrollo de la herramienta y detalla su implementación. En lugar de describir los componentes técnicos de forma aislada, se organiza en torno a los tres flujos principales del sistema, que reflejan los recorridos reales de los datos desde la configuración de un experimento hasta su análisis en el dashboard.

El código fuente completo de este proyecto se encuentra alojado y versionado de forma pública en un repositorio de GitHub, accesible a través del siguiente enlace: <https://github.com/LucasGonzalezPuentes/TFG>.

4.1. Arquitectura del sistema

La herramienta sigue una arquitectura cliente-servidor de tres capas. Cada una tiene una responsabilidad claramente delimitada y se comunica a través de interfaces bien definidas.

- **Capa de presentación:** Aplicación de página única (*Single Page Application*) desarrollada en React que se ejecuta en el navegador del usuario en el puerto 5173. Es la única capa con la que interactúan directamente tanto el evaluador como el usuario participante.
- **Capa de lógica de negocio:** Servidor desarrollado en Python con FastAPI, accesible en el puerto 8000. Expone una API REST que recibe las peticiones del frontend, ejecuta la lógica de procesamiento HCAI y accede a la base de datos mediante el ORM SQLAlchemy.
- **Capa de persistencia:** Instancia de PostgreSQL que almacena de forma persistente todas las entidades del sistema: pruebas, encuestas, eventos de log y métricas objetivas consolidadas.

La comunicación entre el frontend y el backend se realiza mediante peticiones HTTP con cuerpos en formato JSON. Para habilitar esta comunicación entre dos orígenes distintos (puertos 5173 y 8000), el servidor configura una política CORS permisiva durante el desarrollo.

La Figura 4.1 muestra el árbol de directorios del proyecto, estructurado en dos ramas principales bajo la carpeta raíz `tfg-dev`: `frontend` y `backend`.

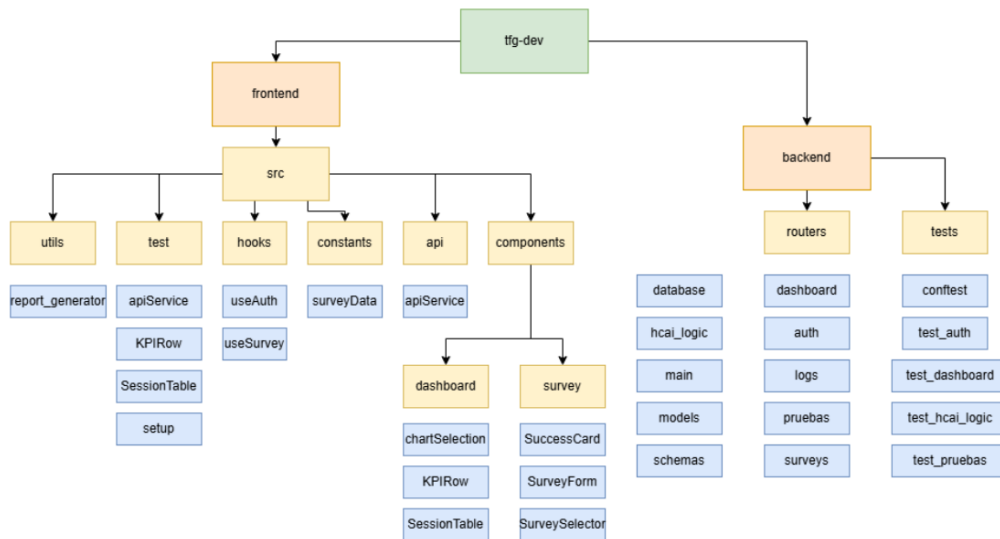


Figura 4.1: Esquema del árbol de directorios con los archivos principales.

4.2. Modelo de datos

La base de datos relacional está compuesta por cuatro tablas, definidas como modelos SQLAlchemy en el backend. Las tablas están relacionadas entre sí mediante sus respectivas FK que garantizan la integridad referencial de los datos a nivel de base de datos. A continuación en la Figura 4.2 se muestra la relación entre las tablas.

- **pruebas:** Almacena cada experimento configurado por el evaluador. Contiene el nombre del sistema bajo prueba, la descripción de la tarea, las métricas del Ground Truth introducidas por el evaluador, el `token_version` único de acceso y la fecha de creación. Es la tabla raíz del sistema: todas las demás entidades se vinculan a ella.
- **encuestas:** Registra las respuestas subjetivas de cada participante. Cada fila almacena el `prueba_id` (FK hacia `pruebas`), el `session_id` que identifica la sesión de forma unívoca, la marca de tiempo del envío y el campo `respuestas` en formato JSON con los valores Likert y NASA-TLX recogidos.
- **eventos_raw:** Contiene cada evento individual registrado durante la interacción del participante con el sistema bajo prueba. Cada fila almacena el `session_id` (FK hacia `encuestas`), el identificador del usuario, el tipo de evento (`task_start`, `interaction`, `error...`), la marca de tiempo y un campo `properties` en JSON con metadatos adicionales como el número de errores.
- **logs_objetivos:** Contiene los datos objetivos consolidados por sesión, calculados durante el procesamiento del log. Almacena el tiempo total de la tarea, el número de clics, los errores cometidos, la tasa de éxito derivada de los eventos y las métricas

técnicas del modelo. Actúa como tabla de resumen que el dashboard consulta para calcular el gap de calibración.

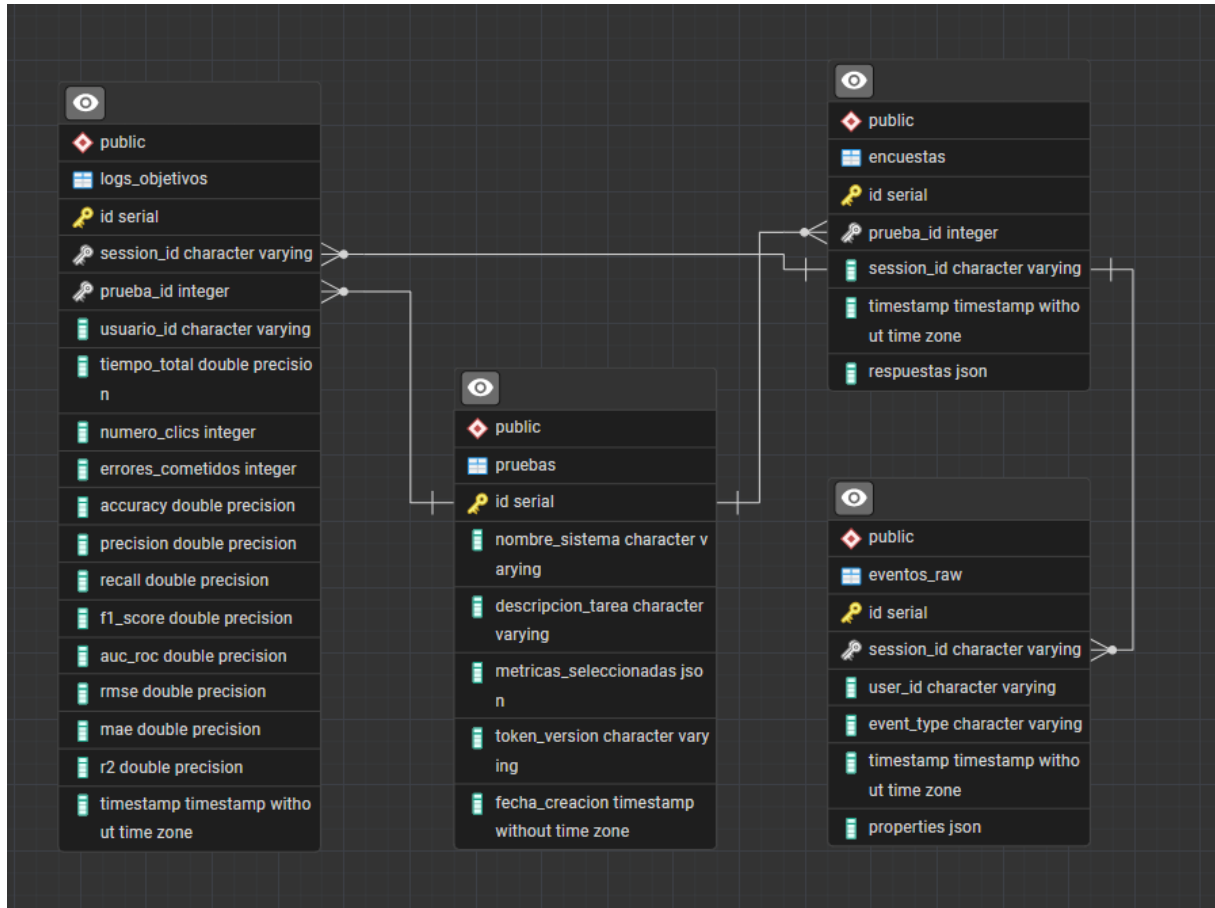


Figura 4.2: Diagrama Entidad-Relación de la base de datos hc.ai_db.

4.3. Estructura del frontend React

El frontend está construido como una *Single Page Application* en React. Para mantener el código organizado y facilitar su mantenimiento, los ficheros se distribuyen en tres categorías principales dentro de la carpeta `src/`:

- **Componentes (`components/`):** Elementos visuales reutilizables que encapsulan tanto la presentación como la lógica local de cada sección de la interfaz. Se dividen en dos subfamilias: los componentes de encuesta (`SurveySelector`, `SurveyForm`, `SuccessCard`), orientados al participante, y los componentes del panel de análisis (`Dashboard`, `EvaluatorPanel`, `KPIRow`, `ChartSection`, `SessionTable`), reservados al evaluador.
- **Hooks (`hooks/`):** Funciones React personalizadas que centralizan la lógica de estado y los efectos secundarios, manteniendo los componentes visuales ligeros. `useSurvey` gestiona todo el ciclo de la encuesta (navegación entre pasos, acumulación de res-

puestas, carga del log y envío al servidor), mientras que `useAuth` controla el estado de autenticación del evaluador y la persistencia del token JWT.

- **Servicios (`api/`):** Módulo `apiService.js` que centraliza todas las llamadas HTTP al backend. Actúa como capa de abstracción entre los componentes y la API REST, evitando que las URLs y los parámetros queden dispersos por el código.

El componente raíz `App.jsx` actúa como enrutador de vistas: en función del estado de autenticación y de la vista activa, renderiza condicionalmente la interfaz pública (encuesta) o las interfaces protegidas (dashboard y configuración). De este modo, un único árbol de componentes sirve a los dos perfiles de usuario sin necesidad de un enrutador externo.

4.4. Interfaz del participante

El participante es el usuario que interactúa con el sistema de IA bajo evaluación y, a continuación, reporta su experiencia a través de la plataforma. Su recorrido está diseñado para ser guiado y minimizar la carga cognitiva: en ningún momento se le pide que tome decisiones técnicas.

4.4.1. Pantalla de selección e inicio de sesión

Al acceder a la aplicación, el participante encuentra una pantalla de bienvenida con un desplegable que lista los experimentos disponibles. Al seleccionar uno, se muestra la descripción de la tarea que el evaluador configuró previamente, informando al participante de qué debe hacer con el sistema externo antes de rellenar la encuesta.

4.4.2. Encuesta multi-paso

Una vez que el participante ha completado la tarea con el sistema externo, regresa a la plataforma y avanza por un asistente de tres pasos, diseñado para segmentar las dimensiones de la evaluación y reducir la fatiga de respuesta, véase un ejemplo de pregunta en la Figura 4.3.

Confianza

1. Tengo confianza en la herramienta. Siento que funciona bien.

- ☐ Completamente de acuerdo
- ☐ Estoy algo de acuerdo
- ☐ Soy neutral al respecto
- ☐ Estoy algo en desacuerdo
- ☐ Completamente en desacuerdo

Figura 4.3: Ejemplo de pregunta incluida en la sección de Confianza.

Al pulsar “Finalizar”, el frontend empaqueta las respuestas, el log y el identificador de sesión en un único objeto y lo envía al servidor. Si el envío es correcto, se muestra una pantalla de confirmación y el sistema queda listo para iniciar una nueva evaluación.

El Listado 4.1 muestra la implementación del endpoint `POST /api/submit-survey`. El uso de `db.flush()` en la Fase 1 es especialmente relevante: permite que el `session_id` de la encuesta quede registrado en la sesión activa y sea accesible como FK por los eventos de la Fase 2, sin necesidad de ejecutar un `commit` prematuro que rompería la atomicidad de la transacción. Si cualquiera de las cuatro fases falla, SQLAlchemy deshace automáticamente todas las operaciones anteriores, garantizando que nunca queden datos parciales en la base de datos.

```

1 @router.post("/submit-survey")
2 def guardar_encuesta_y_log(datos: SubmissionPayload, db: Session = Depends(
  get_db)):
3     # Fase 1 - Encuesta subjetiva
4     db.add(EncuestaLog(prueba_id=prueba.id, session_id=datos.session_id,
5                       respuestas=datos.respuestas))
6     db.flush() # session_id disponible como FK sin commit prematuro
7     # Fase 2 - Eventos raw (acumula errores y tiempo)
8     for evento in datos.log_file:
9         db.add(EventoLog(session_id=datos.session_id, ...))
10        total_errores += evento.properties.get("errors", 0)
11        tiempo_total_ms += evento.properties.get("time_to_complete", 0)
12    # Fase 3 - Consolidacion en logs objetivos
13    accuracy = max(0.0, 1.0 - (total_errores / len(datos.log_file)))
14    db.add(LogObjetivo(session_id=datos.session_id, accuracy=accuracy, ...))
15    db.commit() # confirma las tres fases de forma atomica

```

Listado 4.1: Transacción atómica: el flush permite usar el session_id como FK antes del commit final.

A continuación se detalla el instrumento de evaluación completo empleado en cada paso del asistente.

Sección 1: Confianza en el sistema

Los ítems de esta sección evalúan la confianza percibida por el usuario en el sistema de IA evaluado. Están adaptados de la escala validada de Hoffman et al. [1], empleada también en estudios cuantitativos recientes sobre explicabilidad y confianza en IA [8].

Indica tu grado de acuerdo con cada una de las siguientes afirmaciones sobre el sistema que acabas de utilizar:

Ítem	Enunciado
C1	I am confident in the tool. I feel that it works well.
C2	The outputs of the tool are very predictable.
C3	The tool is very reliable. I can count on it to be correct all the time.
C4	I feel safe that when I rely on the tool I will get the right answers.
C5	The tool is efficient in that it works very quickly.
C6	I am wary of the tool.
C7	The tool can perform the task better than a novice human user.
C8	I like using the system for decision making.

Tabla 4.1: Ítems de la dimensión Confianza (escala Likert 1–5). Adaptados de Hoffman et al. [1].

La escala de respuesta empleada para todos los ítems de las secciones 1 y 2 es la siguiente:

Valor	Etiqueta
5	I agree strongly
4	I agree somewhat
3	I'm neutral about it
2	I disagree somewhat
1	I disagree strongly

Tabla 4.2: Escala de respuesta Likert empleada en las secciones 1 y 2.

Sección 2: Explicabilidad del sistema

Los ítems de esta sección evalúan en qué medida el sistema proporciona explicaciones comprensibles y útiles. Están adaptados del marco COP-12 [9], que cubre las dimensiones de corrección, completitud, coherencia y utilidad contextual de las explicaciones.

Indica tu grado de acuerdo con cada una de las siguientes afirmaciones sobre las explicaciones ofrecidas por el sistema:

Ítem	Enunciado
E1	From the explanation, I know how the tool works.
E2	This explanation of how the tool works is satisfying.
E3	This explanation of how the tool works has sufficient detail.
E4	This explanation of how the tool works seems complete.
E5	This explanation of how the tool works tells me how to use it.
E6	This explanation of how the tool works is useful to my goals.
E7	This explanation of the tool shows me how accurate the tool is.

Tabla 4.3: Ítems de la dimensión Explicabilidad.

Sección 3: Carga cognitiva (NASA-TLX)

Esta sección emplea la escala NASA Task Load Index [2] para medir la carga de trabajo subjetiva experimentada durante la realización de la tarea. Cada dimensión se valora mediante un deslizador continuo de 0 a 100.

Desplaza el indicador para indicar tu valoración de cada dimensión durante la realización de la tarea:

Ítem	Descripción	Extremo bajo	Extremo alto
N1	Mental Demand. How mentally demanding was the task?	Very Low	Very High
N2	Physical Demand. How physically demanding was the task?	Very Low	Very High
N3	Temporal Demand. How hurried or rushed was the pace of the task?	Very Low	Very High
N4	Performance. How successful were you in accomplishing what you were asked to do?	Perfect	Failure
N5	Effort. How hard did you have to work to accomplish your level of performance?	Very Low	Very High
N6	Frustration. How insecure, discouraged, irritated, stressed, and annoyed were you?	Very Low	Very High

Tabla 4.4: Ítems de la dimensión Carga Cognitiva, adaptados de la escala NASA-TLX [2].

4.4.3. Estructura del archivo de log

El archivo de log que el participante adjunta al finalizar la encuesta es un array en formato JSON en el que cada elemento representa un evento registrado durante la interacción con el sistema bajo prueba. Cada evento contiene cuatro campos obligatorios: **event**, que identifica el tipo de acción realizada (**task_start**, **interaction**, **ai_suggestion_accepted**, **manual_override** o **task_end**); **user_id**, que identifica al participante; **session_id**, que vincula el evento con la encuesta correspondiente; y **timestamp**, con la marca temporal exacta del evento.

Cada evento incluye además un objeto **properties** con dos campos que el backend extrae durante el procesamiento: **errors**, que registra el número de errores cometidos en ese evento y que se acumula para calcular el total de errores de la sesión, y **time_to_complete**, que se suma al tiempo total de la tarea y se convierte a segundos al persistir el registro consolidado en la tabla **logs_objetivos**.

El Listado 4.2 muestra un ejemplo representativo del archivo de log esperado por la plataforma.

```
1 [
2   {
3     "event":      "task_start",
4     "user_id":    "usr_participante",
5     "session_id": "sesion_1748392847",
6     "timestamp":  "2025-11-10T10:00:00.000Z",
7     "properties": {
8       "errors":      0,
9       "time_to_complete": 0,
10      "description":  "Usuario inicia la tarea"
11    }
12  }
13 ]
```

Listado 4.2: Ejemplo de archivo de log de un evento registrado durante una sesión de evaluación.

4.4.4. Procesamiento en el servidor

El endpoint **POST /api/submit-survey** procesa la petición en cuatro fases dentro de una única transacción de base de datos, garantizando atomicidad.

Fase 0 — Verificación. El servidor comprueba que el **prueba_id** recibido corresponde a una prueba existente.

Fase 1 — Encuesta. Las respuestas subjetivas se persisten en la tabla **encuestas**. Se ejecuta un **flush** para que el **session_id** quede disponible como FK en la transacción activa sin necesidad de un commit prematuro.

Fase 2 — Eventos. Cada evento del log se inserta individualmente en **eventos_raw**.

Simultáneamente se acumulan el total de errores y el tiempo total de la tarea en milisegundos.

Fase 3 — Consolidación. Con el objetivo de disponer de una medida objetiva del comportamiento del usuario durante la interacción con el sistema bajo prueba, se define la *tasa de éxito* como el complemento de la proporción de errores cometidos sobre el total de eventos registrados en el log.

$$\text{success_rate} = 1 - \frac{\text{total_errores}}{\text{num_eventos}} \quad (4.1)$$

Finalmente, el commit confirma la transacción completa de forma atómica.

4.5. Interfaz del evaluador

El evaluador es el investigador responsable del experimento. Accede a las vistas protegidas de la plataforma mediante autenticación JWT y dispone de dos espacios diferenciados: el panel de configuración y el dashboard de análisis.

4.5.1. Autenticación y control de acceso

El acceso al dashboard y al panel de configuración está protegido mediante JWT. Al enviar credenciales válidas al endpoint `POST /api/login`, el servidor genera y devuelve un token firmado con una clave secreta mediante el algoritmo HS256.

El frontend almacena este token y controla el estado de autenticación mediante una variable de estado. Las vistas del evaluador solo se renderizan si esta variable (`isAuthenticated`) es verdadera; en caso contrario, se muestra el formulario de login. El cierre de sesión elimina el token del almacenamiento local y redirige el usuario a la vista pública de encuesta, que permanece accesible sin autenticación para cualquier participante.

4.5.2. Panel de configuración de experimentos

A través del componente `EvaluatorPanel`, el evaluador define los experimentos que los participantes van a evaluar. El formulario se estructura en tres bloques: el nombre del sistema bajo prueba, la descripción de la tarea que se mostrará al participante, y las métricas del Ground Truth de la IA (Accuracy, Precision, Recall, F1-Score, AUC-ROC, RMSE, MAE y R^2). Cabe destacar que no es obligatorio proporcionar valores para todas estas métricas, ya que el evaluador solo necesita completar aquellas que sean pertinentes para el modelo evaluado.

Al guardar, el backend genera un `token_version` único de ocho caracteres y devuelve el enlace de acceso que el evaluador puede distribuir a los participantes. El identificador numérico (`id`) generado por PostgreSQL actúa como clave interna del sistema, mientras que el `token_version` es el identificador legible orientado al exterior.

4.5.3. Dashboard de análisis

Una vez recogidos los datos de los participantes, el evaluador accede al dashboard para analizarlos. Al cargar la vista, el frontend realiza en paralelo dos peticiones GET: `/api/dashboard-metrics`, que agrega los datos subjetivos y objetivos por sesión, y `/api/log-metrics`, que analiza los eventos en bruto de la tabla `eventos_raw`. El evaluador puede seleccionar el experimento a visualizar desde un desplegable en la cabecera, y exportar un informe HTML autocontenido con todos los resultados mediante el botón “Generar Informe TFG”.

Los resultados se organizan en cuatro bloques visuales:

1. **KPIs principales:** Cuatro tarjetas con la confianza media, el número de sesiones realizadas, la tasa de éxito media derivada del log de interacción y el gap de calibración. La confianza media se expresa en escala 0–100, al igual que la tasa de éxito (multiplicada por 100), permitiendo su comparación directa:

$$\text{gap_calibración} = \text{confianza_media} - \text{success_rate_media} \quad (4.2)$$

Un valor positivo elevado señala sobreconfianza (*overtrust*); uno negativo, infraconfianza (*undertrust*).

2. **Métricas subjetivas:** Un *LineChart* que muestra la evolución de confianza y tasa de éxito por sesión para detectar tendencias temporales, y un *BarChart* de triangulación con los promedios globales de confianza, explicabilidad, carga cognitiva y tasa de éxito.
3. **Métricas objetivas de logs:** Un *BarChart* horizontal con la distribución de tipos de evento, un *BarChart* agrupado con errores e interacciones por sesión, un *ScatterChart* de tiempo de sesión frente a tasa de acierto, y un *BarChart* con las métricas del Ground Truth introducidas por el evaluador.
4. **Ranking e histórico:** Lista de las cinco mejores sesiones ordenadas por puntuación HCAI Score y tabla completa con todas las sesiones desglosadas por métrica individual.

4.5.4. Cálculo del HCAI Score

Las respuestas Likert se normalizan a una escala 0–100 según la Tabla 4.5, y se distribuyen entre las dos dimensiones según su identificador: **p1–p8** contribuyen a la confianza y **p9–p15** a la explicabilidad. Las respuestas NASA-TLX se promedian directamente como carga cognitiva. La puntuación de cada dimensión es la media aritmética de sus ítems.

Opción	Texto	Valor normalizado
a	Completamente de acuerdo	100
b	Estoy algo de acuerdo	75
c	Soy neutral al respecto	50
d	Estoy algo en desacuerdo	25
e	Completamente en desacuerdo	0

Tabla 4.5: Normalización de la escala Likert.

El frontend calcula una puntuación global HCAI por sesión que integra las tres dimensiones en un único indicador, combinando positivamente confianza y explicabilidad e inversamente la carga cognitiva:

$$\text{HCAI_score} = \frac{\text{confianza} + \text{explicabilidad} + (100 - \text{carga_cognitiva})}{3} \quad (4.3)$$

Este valor se emplea para ordenar el ranking de sesiones en el panel de análisis.

Capítulo 5

Evaluación

Este capítulo describe el proceso de validación de la herramienta desarrollada mediante una suite de pruebas automatizadas que cubre tanto el *backend* como el *frontend*. La primera parte detalla la suite de pruebas automatizadas implementada para verificar la corrección de la lógica interna, los endpoints de la API y los componentes de la interfaz de usuario. La segunda parte describe el diseño de una prueba de uso con una aplicación real, orientada a validar la utilidad del sistema en un escenario experimental controlado.

5.1. Suite de pruebas automatizadas

Con el objetivo de garantizar la correcta integración entre las distintas capas del sistema y la fiabilidad de los cálculos HCAI, se ha desarrollado una suite de 120 casos de prueba organizados en siete módulos independientes.

5.1.1. Infraestructura de pruebas

Para aislar las pruebas del entorno de producción, el fichero `conftest.py` sustituye la base de datos PostgreSQL por una instancia SQLite en memoria. Cada test se ejecuta dentro de una transacción que se deshace automáticamente al finalizar, garantizando el aislamiento total sin necesidad de limpiar el estado manualmente. El `TestClient` de FastAPI sobrescribe la dependencia `get_db` para apuntar a esta sesión, de modo que los endpoints se ejercitan de extremo a extremo sin ninguna llamada de red real.

En el *frontend*, las pruebas se ejecutan con **Vitest** y **React Testing Library**, interceptando las peticiones HTTP para sustituirlas por respuestas simuladas.

5.1.2. Módulos de prueba

Lógica HCAI: Pruebas unitarias puras sobre `calcular_score_hcai`, sin base de datos ni red. Se valida la tabla de normalización Likert, los casos límite (diccionario vacío, claves desconocidas, valores NASA-TLX no numéricos, letras fuera de escala), la asignación correcta de preguntas a cada dimensión (confianza para p1–p8, explicabilidad para p9 en adelante, incluyendo claves con sufijo) y el promedio de los ítems NASA-TLX, incluyendo

la conversión de cadenas de texto a flotante, y un caso de encuesta completa que combina las tres dimensiones.

Autenticación: Valida el endpoint de login ante credenciales válidas (HTTP 200 con token JWT de tres segmentos), las cuatro combinaciones de credenciales inválidas (HTTP 401) y cuerpos malformados (HTTP 422).

Gestión de experimentos: Cubre el ciclo de vida completo de una prueba: creación con token de ocho caracteres y enlace de acceso, unicidad de tokens entre pruebas consecutivas, listado con los campos esperados y validación de la comparación A/B, comprobando que dos tokens idénticos devuelven valores simétricos.

Dashboard: Módulo más extenso de la suite. Valida las métricas de dashboard y logs en dos escenarios: base de datos vacía (contadores a cero, `null` ante prueba inexistente) y con datos reales sembrados por `session_fixture`. En este segundo escenario se verifica que la tasa de éxito real se convierte correctamente a porcentaje ($0,80 \rightarrow 80,0\%$), que el *accuracy* se extrae de las métricas del evaluador ($0,85 \rightarrow 85,0\%$) y que los bloques *subjetivo* y *objetivo* contienen todas sus claves.

Capa de servicio: Verifica para cada función de la API que la URL y el método HTTP son correctos, que el payload se serializa adecuadamente y que los errores del servidor lanzan la excepción esperada.

Componentes KPI: Comprueba que las cuatro tarjetas del dashboard se renderizan con sus etiquetas y valores, y que el gap de calibración se calcula correctamente (confianza $80\% - \text{tasa de éxito } 65\% = 15\%$).

Tabla de sesiones: Valida que la cabecera contiene todas las columnas esperadas, que el identificador de sesión se trunca a los últimos ocho caracteres, que los valores nulos se muestran como “—” y que el ranking nunca supera las cinco primeras entradas independientemente del tamaño de la lista.

5.1.3. Resumen de la suite

Módulo	Componente validado	Tests	Tecnología
<code>test_hcai_logic.py</code>	Función <code>calcular_score_hcai</code>	22	pytest
<code>test_auth.py</code>	Endpoint POST <code>/api/login</code>	8	pytest + TestClient
<code>test_pruebas.py</code>	Endpoints de gestión de experimentos	12	pytest + TestClient
<code>test_dashboard.py</code>	Endpoints <code>dashboard-metrics</code> y <code>log-metrics</code>	22	pytest + TestClient
<code>apiService.test.js</code>	Capa de comunicación con la API REST	17	Vitest
<code>KPIRow.test.jsx</code>	Componentes <code>KPICard</code> , <code>KPIRow</code> , <code>LogKPIRow</code>	20	Vitest + RTL
<code>SessionTable.test.jsx</code>	Componentes <code>SessionTable</code> , <code>RankingList</code>	19	Vitest + RTL
Total		120	

Tabla 5.1: Resumen de la suite de pruebas automatizadas.

5.2. Prueba de uso con aplicación real: Predictor de Diabetes

Para validar la utilidad del sistema en un escenario experimental, se diseñó una prueba completa utilizando una aplicación de Inteligencia Artificial real y públicamente accesible. El objetivo de esta prueba no es evaluar la herramienta de predicción de diabetes en sí misma, sino demostrar que la plataforma HCAI desarrollada es capaz de recolectar, procesar y visualizar correctamente los datos de una sesión de evaluación real de principio a fin.

5.2.1. Selección y descripción de la aplicación bajo prueba

La aplicación seleccionada es *Diabetes Prediction*, un espacio publicado en Hugging Face [10] que implementa un modelo de clasificación binaria entrenado sobre el conocido *Pima Indians Diabetes Dataset* [11]. Este conjunto de datos recoge registros clínicos de mujeres de la comunidad indígena Pima con el objetivo de predecir la aparición de diabetes de tipo 2 a partir de variables fisiológicas. La interfaz permite al usuario introducir ocho parámetros clínicos —número de embarazos, nivel de glucosa, presión arterial, grosor del pliegue cutáneo, nivel de insulina, IMC, función de pedigrí de diabetes y edad— y recibe como respuesta una predicción del modelo (véase Figura 5.1). La tarea consiste en una clasificación binaria: dado un conjunto de parámetros clínicos, el modelo predice si el paciente es diabético o no diabético.

E-DiaB

Welcome to this Early Diabetes Prediction Model!

This model serves as a valuable tool for physicians, assisting them in making more accurate predictions about a patient's risk for diabetes, enabling earlier intervention and improved patient care. According to the World Health Organization (WHO), diabetes can often go undetected for several years with warning signs that may be subtle.

By simply entering personalized health metrics, such as blood pressure, age, and BMI, you will receive a customized evaluation of your risk for diabetes. This proactive approach empowers individuals to take informed steps towards managing and preventing diabetes.

Number of Pregnancies

0

Glucose Level

0

Blood Pressure value

0

Skin Thickness value

0

Insulin Level

Figura 5.1: Interfaz de usuario del sistema de predicción de diabetes

Esta aplicación resulta especialmente adecuada para el paradigma HCAI por tres razones. En primer lugar, la tarea implica una decisión con consecuencias percibidas como relevantes por el usuario. En segundo lugar, el sistema ofrece únicamente el resultado de la predicción sin explicación alguna del razonamiento subyacente, lo que permite estudiar el efecto de la ausencia de explicabilidad sobre la confianza del usuario. En tercer lugar, el autor del modelo documenta sus métricas técnicas, lo que permite introducir un *Ground Truth* verificado en el panel del evaluador.

5.2.2. Configuración del experimento en la plataforma

El primer paso consistió en acceder al panel de configuración mediante las credenciales de evaluador y crear una nueva prueba con el nombre *Diabetes Prediction (Hugging Face)*, la descripción de la tarea asignada al participante y las métricas del Ground Truth documentadas por el autor del modelo: Accuracy 0,95 y AUC-ROC 0,90. Al guardar, la plataforma generó automáticamente el `token_version` y el enlace de acceso, confirmando el correcto funcionamiento del flujo de configuración.

Configuración de Experimentos

Define una nueva variante del sistema para comparar resultados en el Dashboard.

1. Nombre de la Versión

E-Diab

2. Objetivo de la Tarea

Este texto se mostrará al usuario antes de comenzar la evaluación.

Usa E-Diab para predecir la aparición de diabetes de tipo 2 a partir de variables fisiológicas.

3. Ground Truth (Métricas Técnicas de la IA)

Introduce los valores reales del sistema evaluado. Aparecerán en el Dashboard para triangulación.

Accuracy (Exactitud)	Precision
0.95	0.0 - 1.0
Recall (Sensibilidad)	F1-Score
0.0 - 1.0	0.0 - 1.0
AUC-ROC	RMSE (Root Mean Sq. Error)
0.90	Valor numérico
MAE (Mean Absolute Error)	R ² (Coef. Determinación)
Valor numérico	0.0 - 1.0

Crear Nueva Prueba

Figura 5.2: Panel de configuración de experimentos: definición de la prueba *E-DiaB* con el Ground Truth introducido por el evaluador.

5.2.3. Flujo de evaluación seguido por el participante

Inicio de sesión. Al seleccionar la prueba y pulsar “Comenzar Evaluación”, el sistema redirige al usuario directamente a la encuesta, como se puede observar en la figura 5.3.

Encuesta subjetiva. Se completaron los tres pasos del asistente: confianza, explicabilidad y carga cognitiva, adjuntando el log descargado del sistema bajo prueba (en este caso datos sintéticos). El frontend empaquetó todo en un único `SubmissionPayload` y lo envió al backend, que confirmó la operación.



Figura 5.3: Pantalla de bienvenida del participante, mostrando la tarea asignada para la prueba *E-DiaB*.

5.2.4. Verificación de los resultados en el dashboard

Tras la sesión, se accedió al dashboard para comprobar la correcta visualización de los datos. Los KPIs mostraron la confianza media, el número de sesiones, la tasa de éxito derivada del log y el gap de calibración. Este último constituyó el resultado más relevante (-57.6 %): la ausencia de explicaciones en la aplicación generó una confianza subjetiva inferior a la tasa de éxito media derivada de los logs de interacción, evidenciando un fenómeno de *undertrust* que ilustra precisamente el tipo de diagnóstico que la herramienta está diseñada para detectar (véase Figura 5.4).



Figura 5.4: Métricas subjetivas por sesión en el dashboard: evolución de Confianza, Explicabilidad y Carga Cognitiva a lo largo de las sesiones registradas.

Con el objetivo de poblar el dashboard con un volumen de datos suficiente para validar el conjunto de visualizaciones, se generaron diez archivos de log sintéticos mediante IA generativa. Cada archivo simula los eventos variados que produciría el sistema bajo prueba durante una sesión real para reproducir perfiles de usuario. Estos logs fueron combinados con respuestas de encuesta completadas manualmente, permitiendo verificar el correcto funcionamiento del ranking, la tabla de sesiones, los gráficos de distribución de eventos y el cálculo del HCAI Score sobre un conjunto de datos representativo.

5.2.5. Conclusiones de la prueba

La prueba de uso demostró que la herramienta es capaz de ejecutar un ciclo de evaluación HCAI completo con una aplicación de IA real. Cabe destacar que, en un escenario de uso real, el sistema bajo prueba debería generar de forma automática el archivo de log en el formato esperado por la plataforma. Dicho esto, el flujo de tres pasos —configuración por parte del evaluador, adjunto de interacciones por parte del sistema bajo prueba y análisis de resultados en el dashboard— funcionó de forma coherente y sin interrupciones.

Asimismo, la prueba permitió identificar un escenario real de *undertrust* asociado a la falta de explicabilidad del modelo evaluado, validando la utilidad práctica del gap de calibración como indicador diagnóstico del paradigma HCAI.

Capítulo 6

Conclusiones

Este capítulo recoge los logros alcanzados a lo largo del proyecto, evalúa el grado de cumplimiento de los objetivos inicialmente propuestos y plantea posibles líneas de trabajo futuro.

6.1. Logros alcanzados

El presente Trabajo de Fin de Grado ha resultado en el diseño e implementación de una plataforma web funcional para la evaluación de sistemas de Inteligencia Artificial desde la perspectiva de la IA Centrada en el Ser Humano. La herramienta integra en un único entorno tres dimensiones que se identifican como complementarias en la evaluación HCAI [4]: la evidencia objetiva derivada de los logs de interacción, la percepción subjetiva recogida mediante cuestionarios y las métricas del *Ground Truth* introducidas por el evaluador, que permiten contrastar el rendimiento real del modelo con la experiencia reportada por los participantes.

La plataforma ha sido construida siguiendo una arquitectura cliente-servidor de tres capas, con un frontend React, un backend FastAPI y una base de datos PostgreSQL, y ha superado una suite de 120 pruebas automatizadas que cubren tanto la lógica de negocio como los componentes de la interfaz. La prueba de uso con el predictor de diabetes de Hugging Face ha demostrado que el sistema es capaz de ejecutar un ciclo de evaluación completo con una aplicación de IA real y sin necesidad de modificar el sistema externo evaluado.

6.2. Grado de cumplimiento de los objetivos

Los objetivos parciales definidos en la introducción se han alcanzado de la siguiente forma:

Definición de un conjunto de métricas HCAI. Se ha establecido un marco de indicadores que combina tres dimensiones subjetivas —confianza, explicabilidad y carga cognitiva— con métricas objetivas derivadas de los logs de interacción. La dimensión de confianza se apoya en la escala validada de Hoffman et al. [1], empleada también en estudios cuantitativos recientes sobre explicabilidad y confianza en IA [8]; la de explicabilidad,

en los ítems del marco COP-12 [9]; y la carga cognitiva, en la escala NASA-TLX [2]. Estos tres instrumentos se integran en el *HCAI Score*, un indicador global que permite comparar sesiones y detectar fenómenos de *overtrust* e *undertrust* a través del gap de calibración.

Desarrollo de un sistema de ingesta y procesamiento de datos. El backend implementa un endpoint transaccional que persiste, en una única operación atómica, las respuestas subjetivas, los eventos de log en bruto y el resumen objetivo consolidado por sesión. La API REST expone además endpoints de agregación para el dashboard y de comparación A/B entre versiones, cubriendo así el ciclo completo desde la recogida hasta el análisis de los datos.

Implementación de visualización interactiva. El dashboard ofrece cuatro bloques de visualización —KPIs, métricas subjetivas, métricas objetivas e histórico de sesiones— implementados con la librería Recharts. El evaluador puede filtrar los resultados por experimento, comparar versiones del sistema bajo prueba mediante gráficos de triangulación y exportar un informe HTML autocontenido, haciendo que los datos brutos se conviertan en información accionable.

Evaluación de la herramienta. La validación se ha llevado a cabo en dos planos complementarios: una suite de pruebas automatizadas que garantiza la corrección de la lógica interna, y una prueba de uso con una aplicación real que verifica la utilidad del sistema en un escenario experimental controlado. La prueba permitió además identificar un caso de *undertrust* asociado a la ausencia de explicabilidad en el modelo evaluado, confirmando la capacidad diagnóstica del gap de calibración.

6.3. Líneas de trabajo futuro

A pesar de los resultados satisfactorios, la herramienta presenta margen de mejora en varias direcciones:

Explicabilidad adaptativa. Los estudios cuantitativos sobre XAI apuntan a que el efecto de las explicaciones sobre la confianza varía significativamente según el perfil del usuario [8]: los usuarios novatos responden mejor a explicaciones narrativas, mientras que los expertos prefieren información técnica detallada. Una línea natural de evolución consistiría en dotar a la plataforma de mecanismos para personalizar la profundidad de las explicaciones en función del nivel de experiencia declarado por el participante.

Métricas de comportamiento objetivas. El sistema actual deriva la tasa de éxito a partir de los errores registrados en el log. Futuros trabajos podrían incorporar indicadores más ricos —como la latencia de respuesta, la tasa de corrección manual o el número de consultas al sistema— que permitirían mostrar la confianza de forma más objetiva, en línea con lo propuesto por Sunny [8] y con los indicadores fisiológicos explorados en la literatura de interacción humano-máquina.

Soporte para experimentos longitudinales. La herramienta está concebida para sesiones únicas. Añadir soporte para el seguimiento de un mismo participante a lo largo del tiempo permitiría estudiar la evolución de la confianza tras exposición repetida al sistema, una dimensión identificada como clave para la calibración de la confianza a largo plazo [12].

Comparación A/B integrada en el dashboard. Aunque el backend expone un endpoint de comparación entre versiones, su integración visual en el dashboard queda como trabajo pendiente. Incorporar un módulo de A/B testing permitiría al evaluador contrastar directamente el impacto de cambios en el sistema de IA sobre las métricas HCAI sin necesidad de consultas manuales a la API.

Apéndice A

Lista de Acrónimos

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
FK	Foreign Key
HCAI	Human-Centered Artificial Intelligence
IA	Artificial Intelligence
IDE	Integrated Development Environment
JWT	JSON Web Token
KPI	Key Performance Indicator
ORM	Object-Relational Mapping
REST	Representational State Transfer
UX	User Experience

Bibliografía

- [1] R. Hoffman, S. Mueller, G. Klein, and J. Litman, “Measuring trust in the xai context.” PsyArXiv Preprints, 2021.
- [2] National Aeronautics and Space Administration (NASA), “Nasa task load index (tlx).” <https://www.nasa.gov/human-systems-integration-division/nasa-task-load-index-tlx/>, 2024. Accedido el 11 de abril de 2026.
- [3] S. Schmager, I. O. Pappas, and P. Vassilakopoulou, “Understanding human-centred ai: a review of its defining elements and a research agenda,” *Behaviour & Information Technology*, vol. 44, no. 15, pp. 3771–3810, 2025.
- [4] G. Desolda, A. Esposito, R. Lanzilotti, A. Piccinno, and M. F. Costabile, “From human-centered to symbiotic artificial intelligence: a focus on medical applications,” *Multimedia Tools and Applications*, vol. 84, pp. 32109–32150, 2025.
- [5] Boletín Oficial del Estado, “Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales,” 2018. Disponible en: <https://www.boe.es/eli/es/lo/2018/12/05/3>.
- [6] I. Sommerville, *Software Engineering*. Pearson, 10th ed., 2016.
- [7] Meta Platforms, Inc., “React documentation - a javascript library for building user interfaces.” <https://react.dev/>, 2026.
- [8] A. D. Sunny, “Preliminary quantitative study on explainability and trust in AI systems.” arXiv preprint arXiv:2510.15769, 2025.
- [9] M. Nauta *et al.*, “From anecdotal evidence to quantitative evaluation methods: A systematic review on evaluating explainable AI,” *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–42, 2023.
- [10] Jaameyaw, “Diabetes prediction.” Hugging Face Spaces, 2024. Consultado: junio 2026.
- [11] J. Smith, J. Everhart, W. Dickson, W. Knowler, and R. Johannes, “Pima indians diabetes dataset.” UCI Machine Learning Repository, 1988.
- [12] A. Jacovi, A. Marasovic, T. Miller, and Y. Goldberg, “Formalizing trust in artificial intelligence: Prerequisites, causes and goals of human trust in AI.” arXiv preprint arXiv:2012.04612, 2021.